

Test Report -- DL-06 Persistence, Supplier Learning, Scheduler, Stats

Branch: feat/dl06-persistence-learning Date: 2026-05-31 How it was tested: automated calls against the live server (running on port 8000)

What was tested

DL-06 added four things, and this report checks each one. First, that the agent log and the approval queue are saved to a small database and come back unchanged after the server is restarted. Second, that a supplier's reliability score moves the right way after a delivery -- up when it arrives on time, down when it is late. Third, that the scheduler starts with the server and reports when the next automatic run is due. Fourth, that the stats the dashboard shows have the right shape and the right numbers.

Test results

All nine checks passed, including both score-formula checks.

#	Test	Expected	Result	Pass
T1	Server healthy, scheduler running	scheduler active, next run set	both present	pass
T2	A new log entry is saved to the database	row appears in the agent-log table	saved	pass
T3	A new approval is saved to the database	row appears in the approvals table	saved, pending	pass
T4	Restart the server, data reloads	7 log entries + 1 pending approval	all reloaded, nothing missing	pass
T5a	On-time delivery raises the score	$0.9700 > 0.9760$	0.9760, matches the formula	pass
T5b	A 5-day-late delivery lowers the score	$0.9760 > 0.8808$	0.8808, matches the formula	pass
T6	The stats reading has the right shape	all fields present and correct	all present, values correct	pass
T7	Approving an item updates the database	marked approved, queue empties	approved, queue no longer empty	pass
T8	Resolving the same item twice is refused	refused (400)	refused (400)	pass

How the supplier score is checked

The score uses an exponential moving average -- each new delivery counts for 20%, and the existing score counts for the remaining 80%. A delivery's performance is 1.0 when it is on time and drops by 0.1 for each day late, so a delivery ten days late counts as 0.0.

Worked through on PrecisionParts GmbH, starting from a score of 0.97:

Event	Days late	Performance	Calculation	Before	After	Match
On-time delivery	0	1.0	$0.2 \times 1.0 + 0.8 \times 0.9700$	0.9700	0.9760	pass

Late delivery | 5 | 0.5 | $0.2 \times 0.5 + 0.8 \times 0.9760$ | 0.9760 | 0.8808 | pass

Both results match the formula exactly, to four decimal places.

Persistence

Before the restart the system held seven agent-log entries (including the score-update events and a test entry) and one pending approval worth ?5,500. After the server was killed and started fresh, all seven entries and the one pending approval came back from the database, with nothing lost. The database file is created at the project root the first time the server starts.

The scheduler

The scheduler reports as active as soon as the server starts, and it shows the time of the next automatic run. By default a run fires every 30 minutes; that interval can be changed with an environment variable, and setting it to one minute is a quick way to watch an automatic run happen during testing.

The stats the dashboard shows

The stats reading returns the numbers the dashboard needs: how many decisions have been made, how many were automatic versus human, how many score updates have happened, the autonomous rate, how many approvals are waiting, the value sitting in the queue, the last run, whether the scheduler is active, and when the next run is due. The dashboard turns these into five cards -- decisions made, autonomous rate, awaiting your OK, value in queue, and next auto-run -- and refreshes them every five seconds with the rest of the page.

Known limitations (also noted in DL-06)

Supplier scores are remembered only for the length of a session. The score updates apply to the suppliers held in memory, which are loaded fresh from the seed file each time the server starts, so a restart returns them to their baseline. The learning is real and visible within a session; saving the learned scores to the database is a later step.

There is also no locking between runs. If a scheduled run and a manual run overlap, both can add approvals at the same time. And because a scenario reset reloads the suppliers from the seed file, it undoes any score changes made during the session -- so score tests and scenario injection should not be mixed in the same flow.